Rubrics as Visual-Repair Context for Self-Evolving UI-to-Code Generation

Tianyi Xiong[1]*, Zhengyuan Yang[2], Xiaofei Wang[2], Chung-Ching Lin[2], Ruichun Ma[2], Kevin Lin[2], Zhendong Wang[2], Linjie Li[2], Chenxi Liu[1], Ruibo Chen[1], Ramani Duraiswami[1], Heng Huang[1], Lijuan Wang[2]

[1]University of Maryland, College Park, [2]Microsoft
txiong23@umd.edu

Abstract

Large vision-language models have shown strong progress in UI-to-code generation, yet their test-time self-evolution remains unstable. We first identify a fundamental obstacle, termed *visual repair coupling*: a local code edit may propagate through layout, style, and component dependencies, correcting one visual mismatch while degrading regions that were previously faithful. To address this issue, we present RubSE, a **Rub**ric-guided **S**elf-**E**volution framework that uses rubrics to represent visual feedback as a structured visual-repair context. At each refinement round, RubSE generates typed candidate rubrics, selects one prioritized repair target, and stores previously selected rubrics as history, thereby steering each revision toward a well-scoped visual repair while discouraging repeated or over-broad changes. Evaluations across six VLMs and three UI-to-code benchmarks demonstrate that RubSE substantially outperforms naïve self-evolution in final-round and best-round settings, achieving more stable refinement trajectories and a higher trajectory-level performance ceiling. Further analysis shows that RubSE mitigates trajectory collapse by improving recovery from severe visual regressions, and that stronger rubric generators can transfer effective visual-repair guidance to weaker code improvers.

1 Introduction

Recent progress in large vision-language models (Hurst et al., 2024; Chen et al., 2024; Bai et al., 2025; Chen et al., 2025a; Li et al., 2024) has expanded their role from answering questions (Liu et al., 2023; Chen et al., 2023; Wang et al., 2025b; Xiong et al., 2025b) about images to producing structured, executable outputs grounded in visual inputs (Xiong et al., 2026; Azzolini et al., 2025; Ni et al., 2025a; Zhao et al., 2025a; Yu et al., 2025).

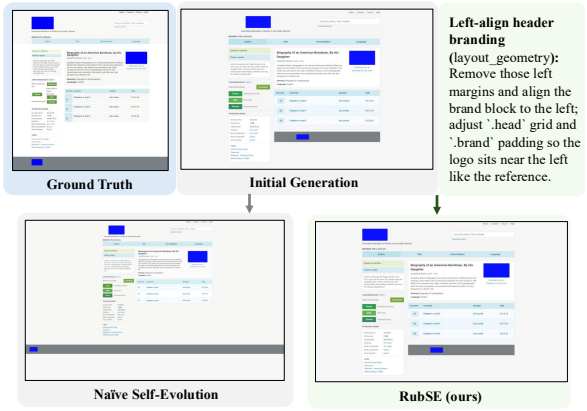


Figure 1: Example of visual repair coupling. Naïve self-evolution fixes the gray bar assignment, but introduces errors in page scale and bar length. In contrast, RubSE correctly left-aligns the logo and tagline with the content edge while preserving the global layout.

UI-to-code generation (Beltramelli, 2018; Jiang et al., 2025; Chen et al., 2025b; Zhou et al., 2025) represents a concrete instance of this shift: given a screenshot, the model must generate web code that renders into a visually faithful reproduction of the target interface. This capability supports UI reproduction, design prototyping, and developer workflows by converting visual designs into executable implementation drafts. Despite its value, UI-to-code generation remains challenging because the quality of the generated code cannot be reliably judged from the code tokens alone. Instead, it must be rendered and evaluated against the target screenshot, requiring the model to align the resulting interface across multiple visual aspects (Si et al., 2025), including spatial layout, color, typography, component hierarchy, and visual density.

Existing methods have primarily improved performance through training-time interventions, such as scaling instruction-tuning data (Gui et al., 2025a; Yun et al., 2024; Calò and De Russis, 2026) or adopting stage-wise training pipelines (Ni et al., 2025a; Yang et al., 2026; Zhao et al., 2025b; Xu et al., 2025). In comparison, test-time improvement

*Work done during an internship at Microsoft.
Project: https://tyxiong23.github.io/rubse

remains less explored. This gap is notable because UI-to-code provides a natural source of test-time feedback: after a code draft is generated, it can be rendered and visually compared with the target screenshot. A straightforward strategy is therefore self-evolution, where the model iteratively critiques the rendered output and revises the code based on observed discrepancies. Such generate–critique–revise loops have been widely studied for text-generation tasks (Huang et al., 2023; Gou et al., 2024; Khattab et al., 2023; Zhang et al., 2025c,b; Liu et al., 2026), and are increasingly supported by general-purpose coding agents (Anthropic, 2025a; OpenAI, 2025a) (see Appendix E for discussion). However, in UI-to-code generation, naïve self-evolution does not reliably produce monotonic improvement.

The key obstacle is that UI code is visually coupled. Changing one CSS rule may shift the layout; modifying a parent container may affect its children; and adding a missing component may alter spacing, alignment, or visual density elsewhere. We refer to this phenomenon as **visual repair coupling**: code edits can induce non-local changes in the rendered interface through coupled layout, style, and component dependencies. Although we study this phenomenon in web UI coding, visual repair coupling reflects a broader challenge in structure–detail generation and agentic tasks, where local edits to implementation details can propagate through the global structure of the final artifact. As illustrated in Figure 1, naïve self-evolution is vulnerable to this coupling. An attempted repair may correct one visible error while damaging regions that were already faithful. Our preliminary analysis suggests that this leads to unstable refinement trajectories, where later iterations do not consistently improve over the initial draft and may even reduce overall visual fidelity.

This failure mode reveals a limitation of free-form visual feedback. A useful test-time critic should not merely tell the model what is wrong; it should also help constrain the repair so that correct regions are not unnecessarily disturbed. In other words, UI-to-code self-evolution must control both sides of visual repair: what should be changed and what should remain preserved. This requires feedback that is structured, localized, and persistent across iterations, rather than an unconstrained critique that may encourage broad code rewrites.

To this end, we propose RubSE, a rubric-guided self-evolution framework for UI-to-code generation. The central idea is to represent visual feedback as structured repair context. Each rubric describes a specific visual failure, assigns it to a predefined visual aspect, and specifies a targeted correction direction. Compared with free-form critique, this rubric-based representation provides soft control over the refinement process: it makes the repair objective explicit while allowing the code generator to determine how to implement the edit. To construct effective repair context across iterations, RubSE uses three atomic operations: EVOLVE, SELECT, and HISTORY. EVOLVE generates candidate rubrics that describe possible visual repairs for the current rendering. SELECT chooses a single prioritized rubric as the focus of the next revision, reducing the risk of broad, entangled edits. HISTORY carries selected rubrics forward across iterations, reminding the model of previous repair decisions and discouraging repeated failures or regressions. Together, these operations separate three roles that are often conflated in naïve self-evolution—discovering errors, choosing a focused repair target, and preserving repair context over time—retaining the flexibility of open-ended code revision while steering the model toward more localized and stable visual improvements.

Across six models and three benchmarks, RubSE improves both metrics over naïve self-evolution in 15/18 final-round settings, with average gains of +1.20 overall points and +0.11 aspect-mean score. It also achieves stronger best-round performance in 14/18 settings, with an average gain of +1.13 overall points, indicating both more stable refinement and a higher trajectory-level ceiling. Additional analysis reveals two key findings: (1) Rubric-guided repair stabilizes self-evolution by accelerating recovery from severe visual regressions and reducing collapse for stronger VLM executors. (2) High-quality rubrics transfer to weaker code improvers, as GPT-5.4-generated rubrics consistently improve Qwen self-evolution by providing perceptual, actionable, and well-scoped repair targets.

Our contributions are summarized as follows:

- We identify *visual repair coupling*, where local code edits induce non-local rendered changes, as a central failure mode behind unstable UI-to-code self-evolution.
- We propose RubSE, a rubric-guided self-evolution framework that turns visual feedback into structured visual-repair context through an EVOLVE–SELECT–HISTORY loop over typed

rubrics, steering the model toward targeted and stable visual improvements.

- Experiments demonstrate that RubSE improves both final-round stability and best-round performance over naïve self-evolution across benchmarks, execution models and metrics. Further analyses show that rubrics help recover from severe visual regressions and provide transferable guidance to weaker code improvers.

2 Related Work

UI-to-Code Generation. Early task-specific methods train encoder–decoder architectures to translate GUI screenshots into intermediate UI descriptions (Beltramelli, 2018) or HTML code (Soselia et al., 2023). Recent advances in vision-language models have enabled more flexible generation from visual inputs (Ge et al., 2025; Jiang et al., 2025; Zhang et al., 2025a; Liu et al., 2025a; Ni et al., 2025b). A line of work constructs large-scale benchmarks from real-world or synthetic webpages. Design2Code (Si et al., 2025) introduces the first real-world benchmark and designs matching- and model-based evaluation metrics. Subsequent works extend this direction through improved data curation pipelines (Yun et al., 2024), broader programming languages (Ge et al., 2025), and fine-grained generation subtasks (Lin et al., 2025).

Another line focuses on improving UI-to-code models. Bhathal and Gupta (2025); Gui et al. (2025a); Jiang et al. (2025); Calò and De Russis (2026) construct high-quality image–code pairs as instruction-tuning datasets. Method-wise, prior research filters self-generated UI programs using compiler and multimodal feedback (Wu et al., 2024), decomposes generation into hierarchy prediction and code synthesis (Gui et al., 2025c), and performs layout-aware reasoning and code assembly for better layout preservation (Gui et al., 2025b). Recent studies further adapt reinforcement learning with verifiable rewards (RLVR, Guo et al. (2025)) to UI-to-code generation task, by applying relative visual-quality signals as reference-based rewards (Yang et al., 2026; Deng et al., 2026). Ni et al. (2025a) introduces an iterative self-debugging loop to improve generation quality, yet the role of structured context for self-improvement remains less explored. Our work studies what context enables test-time self-evolution, guiding targeted fixes while preserving overall visual fidelity.

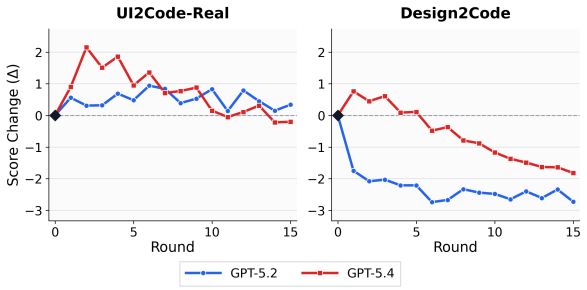


Figure 2: Score delta relative to the initial generation under naïve self-refinement. Both models show diminishing gains and later degradation; on Design2Code, refinement falls below the initial generation.

Rubrics in LLM/VLMs. Rubrics have been widely adopted as structured evaluation criteria for LLM-as-a-Judge systems (Zheng et al., 2023; Zhang et al., 2023), particularly in open-ended domains where holistic reward signals are often insufficient, such as healthcare (Arora et al., 2025) and academic tasks (Akyürek et al., 2025; Yifei et al., 2025; Wang et al., 2025c; Xiong et al., 2025a). By decomposing evaluation into criterion-level decisions, rubrics improve the reliability of model-based judgment and better align automated scores with human evaluation (Wang et al., 2025c; Srivastava et al., 2025; Liu et al., 2025b; Wang et al., 2026). Rubrics have also been utilized as reward, extending RL post-training beyond domains with verifiable final answers. Rubric-based rewards provide fine-grained, instance-specific feedback, improving reasoning trajectories (Yuan et al., 2025) and open-ended task performance (Gunjal et al., 2025; Shao et al., 2025; Rezaei et al., 2025). In contrast, our work treats rubrics as reusable visual repair context: they identify localized visual mismatches, guide subsequent code edits, and accumulate across refinement rounds to steer iterative UI revision toward the reference design.

3 Method

3.1 Task Formulation

We formalize self-evolution for UI-to-code generation as sequential repair. Let x denote the target UI screenshot, and let $R(\cdot)$ be a deterministic rendering function implemented by a headless browser. A VLM generator G_θ first produces an initial code draft $c_0 = G_\theta(x)$, whose rendering is $r_0 = R(c_0)$. At each refinement round $t \geq 1$, a feedback interface $\mathcal{I}$ constructs a repair context

$$\phi_t \;=\; \mathcal{I}\big(x,\, c_{t-1},\, r_{t-1},\, H_{t-1}\big),$$

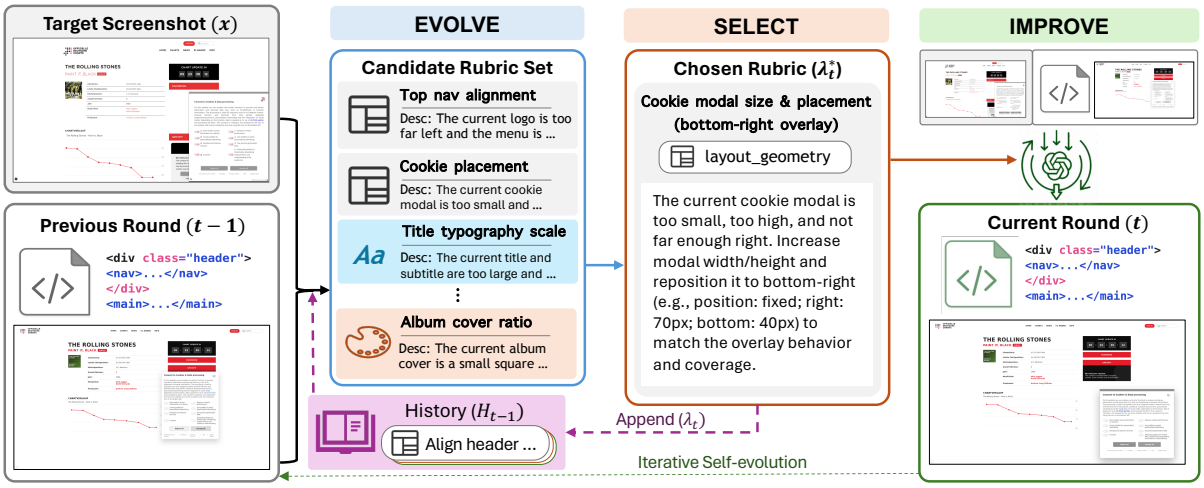


Figure 3: Self-evolution process of RubSE. Given the target screenshot, the previous-round code and rendering, and the accumulated rubric history, RubSE first generates a set of candidate rubrics for remaining visual mismatches. It then selects the most crucial rubric—position and scale of the cookie modal in this example—as a localized repair target, and uses it as context for VLM code revision. After each round, the selected rubric is appended to the history and reused in subsequent rounds, enabling iterative refinement while reducing repeated or uncontrolled edits.

where H_{t-1} summarizes an optional per-instance history of previous repair steps. The generator then refines the previous code based on this context: $c_t = G_\theta(x, c_{t-1}, r_{t-1}, \phi_t)$. The goal is a faithful reconstruction c_T such that $R(c_T)$ visually matches x. Different self-evolution methods differ in how they construct the repair context ϕ_t; Section 3.3 presents our rubric-based instantiation.

3.2 Visual Repair Coupling

Let $\Delta(c, c')$ denote the textual difference between two code states, and let $\Delta(R(c), R(c'))$ denote the rendered-pixel difference between their renderings. In ordinary text generation, edits and outputs are coextensive: $\Delta(c, c')$ *is* the change. In visual code generation, the rendering map R is *non-local*: editing line i of c can change various aspects of $R(c)$ through cascading layout, flex/grid sizing, font metric, and asset placement effects. We call this property *visual repair coupling*.

In a preliminary study, we apply GPT-5.2 and GPT-5.4 to iteratively refine their generated code to match the target screenshot for $N = 15$ rounds. Results are shown in Figure 2. On UI2Code-Real, early gains are small or transient: GPT-5.2 reaches a modest peak improvement of $+0.94$ judge points and finishes at $+0.34$, while GPT-5.4 peaks higher at $+2.15$ by round 2 but then declines below the initial generation (-0.20). The failure mode is stronger on Design2Code: all $15/15$ GPT-5.2 refined rounds are below round 0, ending at -2.73, while GPT-5.4 shows only a short-lived $+0.76$ gain in the first round before declining to -1.82, with $10/15$ rounds below the initial generation. A representative example in Figure 1 illustrates this drift: the model attempts to fix the parent assignment of the bottom gray bar, but instead introduces additional errors in page scale, bar length, and the background color of the bottom-left container, turning a repair attempt into a broader visual degradation.

3.3 RubSE

The results above suggest that effective self-evolution requires a structured visual-repair context. Such context should not only identify what is wrong, but also constrain the scope of the next edit, so that a local repair does not unnecessarily perturb already faithful regions.

In this section, we introduce RubSE, which instantiates each round's repair context with a single-step visual rubric. Rather than imposing hard constraints on the generated code, RubSE takes rubrics as explicit conditioning signals. Each rubric names a targeted visual mismatch, specifies the intended correction, and focuses the current refinement round on a single prioritized repair target, thereby allowing the generator to revise the code flexibly yet with localized guidance.

Formally, we define a visual-repair rubric as a structured object

$$\lambda = \langle \texttt{title}, \texttt{type}, \texttt{description} \rangle,$$

where `description` specifies what is currently mismatched and how it should be corrected, and `type` is drawn from a fixed five-aspect taxonomy: LAYOUT_GEOMETRY, SPACING_DENSITY,

TYPOGRAPHY_TEXT, STYLING_VISUAL, COMPLETENESS. The taxonomy is enforced during candidate generation so that the candidate set spans visually distinct dimensions rather than collapsing onto the most salient defect (e.g. a single bright colour mismatch). Items with malformed JSON or invalid `type` are discarded.

We organize the feedback interface $\mathcal{I}$ around three atomic operations on rubrics to construct the repair context ϕ_t, where the first two are implemented as separate VLM calls:

EVOLVE $(x, r_{t-1}, c_{t-1}, H_{t-1}) \to \Lambda_t$**.** Produces a typed candidate set of K rubrics for the current state. When the cross-round history H_{t-1} is non-empty, it is passed as an explicit avoid list, and EVOLVE is instructed to return only rubrics whose issues are *not* already covered.

SELECT $(x, r_{t-1}, c_{t-1}, \Lambda_t) \to \lambda_t^\star$**.** Selects a single rubric from Λ_t as $\lambda_t^\star$, choosing the one expected to maximize visual improvement under a targeted edit. The selected rubric defines the current repair context, i.e., $\phi_t = \lambda_t^\star$.

HISTORY $(H_{t-1}, \lambda_t^\star) \to H_t = H_{t-1} \cup \{\lambda_t^\star\}$**.** Appends the selected rubric to a per-instance, append-only history, consumed by the next round's EVOLVE as the avoid list.

Given this scoped visual-repair context, the generator emits the next HTML/CSS code conditioned on the target screenshot, the previous state (c_{t-1}, r_{t-1}), and the selected rubric:

$$c_t = G_\theta(x, c_{t-1}, r_{t-1}, \phi_t), \quad \text{where } \phi_t = \lambda_t^\star.$$

As illustrated in Figure 3, each round first obtains a selected rubric through EVOLVE $\to$ SELECT, then uses it to refine the code and appends it to the history via HISTORY. Detailed evolution prompts are provided in Appendix D. Keeping EVOLVE and SELECT as distinct steps is intentional: the former externalizes a typed set of candidate repair targets, while the latter prioritizes one of them under a constrained edit budget, so that prioritization is not entangled with candidate generation in a single pass. This separation design is validated in Appendix B.3.

4 Experiments

4.1 Settings

Implementation Details. We evaluate RubSE on two groups of VLMs: 1) for frontier VLMs, we use GPT-5.4 (OpenAI, 2026), GPT-5.2 (OpenAI, 2025b), and Claude-Sonnet-4.5 (Anthropic, 2025b); 2) for open-source models, we evaluate three leading Qwen (Bai et al., 2025) variants: Qwen3-VL-32B-Instruct, Qwen3.5-9B, and Qwen-3.6-35B-A3B. Unless otherwise specified, all self-evolution methods are run for 10 refinement rounds. For all models, we follow their default inference hyperparameters and set the maximum generation tokens to 32,768. Open-source-model experiments use vLLM v0.19.0 and four NVIDIA H200 GPUs.

Benchmarks. Design2Code (Si et al., 2025) is a widely adopted UI-to-code benchmark containing webpages with diverse structures and layouts, where external images are replaced with placeholders to focus evaluation on reproducible HTML/CSS elements. We evaluate on both the normal (484 samples) and hard (80 samples) subsets. UI2Code-Real (Yang et al., 2026) contains 115 real-world webpages collected from in-the-wild sources, allowing us to assess performance under more challenging and realistic UI complexity.

Evaluation Protocol. Following Yang et al. (2026), we evaluate visual fidelity by using a frontier VLM to compare each rendered webpage image against the target screenshot. We report two metrics: 1) an overall score on a 0–100 scale, and 2) an aspect-level score by averaging 1–7 Likert-scale ratings over five predefined visual aspects. In experiment, we apply GPT-5.2 as the judge model and, for each generated rendering and metric, average over three independent judge runs. Additional protocol details are provided in Appendix A, and judge variability is analyzed in Appendix B.2.

4.2 Main Results

We compare RubSE with direct initial code generation and naïve self-evolution. For iterative methods, we report both the final-round performance and the best round selected by the *overall* score, with the aspect score taken from the same round, as an oracle-performance reference. Results are summarized in Table 1.

RubSE stabilizes self-evolution and improves code generation fidelity. At round 10, RubSE improves over naïve self-evolution by an average of $+1.20$ overall score and $+0.11$ aspect-mean score across the 18 model–benchmark settings. The effect is clearest on frontier executors ($+1.90$ judge, $+0.16$ aspect), where unconstrained revision often

Model	Method	UI2Code-Real		Design2Code		Design2Code-HARD	
		Overall ↑	Aspect ↑	Overall ↑	Aspect ↑	Overall ↑	Aspect ↑
GPT-5.4	Direct	85.4	5.32	86.9	5.63	79.8	4.72
	+ Self-Evolve (r=10)	85.6	5.43	85.8	5.50	78.0	4.61
	+ Self-Evolve (best)	87.6_{r2}	5.61	87.7_{r1}	5.68	80.0_{r1}	4.77
	+ RubSE (r=10)	87.9	**5.66**	87.8	5.76	80.3	4.74
	+ RubSE (best)	**88.1**$_{r8}$	5.63	**88.2**$_{r3}$	**5.79**	**80.8**$_{r4}$	**4.79**
GPT-5.2	Direct	81.6	5.03	86.4	5.59	82.7	5.03
	+ Self-Evolve (r=10)	82.5	5.06	83.9	5.12	82.3	4.92
	+ Self-Evolve (best)	82.6_{r6}	5.06	84.6_{r1}	5.20	82.8_{r2}	4.97
	+ RubSE (r=10)	85.0	5.29	87.2	5.43	84.4	5.13
	+ RubSE (best)	**85.1**$_{r7}$	**5.35**	**88.1**$_{r2}$	**5.71**	**84.8**$_{r7}$	**5.15**
Claude-Sonnet-4.5	Direct	73.3	4.45	83.0	4.98	79.8	4.80
	+ Self-Evolve (r=10)	75.5	4.70	83.3	5.04	80.1	**4.87**
	+ Self-Evolve (best)	75.8_{r3}	4.69	**83.5**$_{r3}$	5.04	**81.2**$_{r8}$	4.84
	+ RubSE (r=10)	77.6	**4.78**	**83.5**	**5.05**	80.4	4.84
	+ RubSE (best)	**79.1**$_{r7}$	4.74	**83.5**$_{r10}$	**5.05**	**81.2**$_{r3}$	4.82
Qwen3-VL-32B	Direct	66.2	4.10	79.6	4.84	78.0	4.74
	+ Self-Evolve (r=10)	**68.6**	**4.23**	81.3	4.95	**84.4**	4.99
	+ Self-Evolve (best)	**68.6**$_{r10}$	**4.23**	81.4_{r7}	4.94	**84.4**$_{r10}$	4.99
	+ RubSE (r=10)	68.3	4.18	81.4	4.96	82.0	**5.04**
	+ RubSE (best)	68.5_{r5}	4.19	**82.0**$_{r5}$	**4.97**	82.8_{r1}	4.99
Qwen3.5-9B	Direct	67.9	4.18	70.3	4.27	72.0	4.36
	+ Self-Evolve (r=10)	69.8	4.28	73.6	4.47	76.0	4.53
	+ Self-Evolve (best)	69.8_{r10}	4.28	74.3_{r7}	4.47	76.0_{r10}	4.53
	+ RubSE (r=10)	70.5	4.34	75.0	**4.56**	**77.6**	**4.64**
	+ RubSE (best)	**70.9**$_{r9}$	**4.35**	**75.6**$_{r7}$	4.55	**77.6**$_{r10}$	**4.64**
Qwen3.6-35B-A3B	Direct	73.3	4.52	80.6	4.95	81.3	4.93
	+ Self-Evolve (r=10)	75.1	4.61	81.6	5.04	82.5	5.01
	+ Self-Evolve (best)	75.5_{r9}	4.58	81.8_{r8}	5.04	82.5_{r10}	5.01
	+ RubSE (r=10)	**77.0**	**4.80**	**82.0**	**5.10**	83.6	5.05
	+ RubSE (best)	**77.0**$_{r10}$	**4.80**	**82.0**$_{r10}$	**5.10**	**85.2**$_{r7}$	**5.14**

Table 1: Self-evolution results across models and benchmarks. We report the overall judge score (0–100) and the mean aspect score (1–7 Likert). "Direct" denotes the initial code generation round. "Self-Evolve" denotes naïve self-evolution. Numbers are reported for the final round-10 result and the best intermediate round selected by the "Overall" score; the subscript indicates the selected round. For each model, the best result in each column is shown in **bold**, and the second-best is underlined.

loses early gains. Open-source executors also benefit: averaged over the 9 Qwen settings, RubSE remains $+0.50$ overall score and $+0.06$ aspect score higher. At the selected best rounds, RubSE outperforms naïve self-evolution on both metrics in $14/18$ settings. This indicates that RubSE improves fixed-budget stability while also providing a higher upper bound for iterative self-evolution across different models and benchmarks. A pairwise human evaluation on 60 UI2Code-Real samples provides complementary evidence that these gains are perceptible: the round-10 outputs of RubSE are preferred over those of naïve self-evolution for both GPT-5.2 and GPT-5.4 ($p < 0.01$; Appendix B.1).

RubSE encourages long-tail iterative improvements on frontier models. On frontier VLMs, RubSE reaches its best checkpoint substantially later than naïve self-evolution on average (r=5.7 vs. r=3.0), while also yielding a larger gain over direct generation ($+2.22$ vs. $+0.77$ overall points). This pattern supports the intended role of rubric history: each round can target a remaining long-tail visual mismatch while preserving previously repaired regions, allowing frontier models to continue exploring long-tail and localized refinements after free-form self-evolution has plateaued or drifted.

Open-source models show larger headroom but heterogeneous dynamics. RubSE improves over direct generation in all 9 Qwen settings, with an average best-round overall gain of $+3.60$ points versus $+2.79$ for naïve self-evolution. For Qwen3.5-9B and Qwen3.6-35B-A3B, RubSE consistently

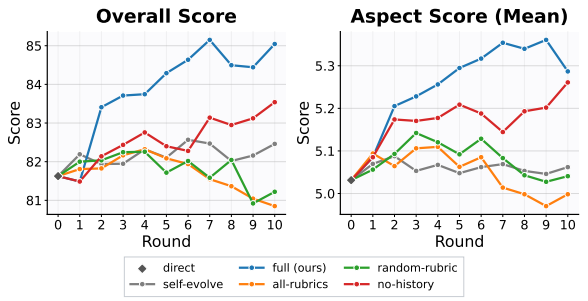


Figure 4: Ablation study on UI2Code-Real with GPT-5.2 as the self-evolution backbone. The full RubSE achieves the overall strongest and most consistent gains across both evaluation metrics.

excels in both final-round and best-round results across metrics and benchmarks. Qwen3-VL-32B-Instruct exhibits mixed results: naïve self-evolution achieves higher overall scores on UI2Code-Real and Design2Code-HARD, while RubSE wins on Design2Code. As discussed in Appendix B.4, this behavior stems from a model-specific imbalance in rubric generation and selection: Qwen3-VL-32B-Instruct overemphasizes localized spacing-density adjustments while underweighting completeness issues. This suggests that typed rubrics are broadly useful for open-source executors, although their effectiveness is model-dependent.

Computational Cost. Compared with naïve self-evolution, RubSE incurs 1.60× the API cost with GPT-5.4 and 2.5% higher inference latency with Qwen3.5-9B under standard vLLM serving. A detailed cost breakdown is provided in Appendix B.3.

5 Analysis

5.1 Ablation Study

We conduct an ablation study on UI2Code-Real using GPT-5.2 as the VLM for self-evolution. As shown in Figure 4, the full method achieves the strongest and most consistent improvements across both overall and aspect-level metrics. Using all generated rubrics or a randomly selected one yields modest gains over naïve self-evolution in early rounds, but the gains are unstable and may drop in later iterations, confirming that selection helps control the edit scope toward the most crucial yet localized visual issue. Removing rubric history leads to consistent but smaller improvements, indicating that past rubrics reduce repeated local repairs and encourage exploration across visual dimensions.

Model	Self-Evolve		RubSE (ours)	
	Collapse ↓	Recover ↑	Collapse ↓	Recover ↑
GPT-5.2	20.8%	26.2%	13.5%	24.2%
GPT-5.4	17.9%	18.5%	11.2%	50.0%
Claude-4.5	18.0%	17.3%	13.8%	24.1%
Qwen3-VL	11.5%	23.1%	15.3%	22.6%
Qwen3.5	17.9%	21.2%	19.4%	23.5%
Qwen3.6	14.9%	26.2%	19.6%	44.4%

Table 2: Trajectory collapse and recovery rates. Results are aggregated over all three benchmarks.

5.2 How RubSE Mitigates Trajectory Collapse?

We define a *visual-repair collapse* as a consecutive-round regression where the overall score drops by at least 8 points and the aspect-mean score drops by at least 0.35. A recovery occurs when a later round returns close to the pre-collapse state, within 2 overall points and 0.10 aspect-mean. A trajectory is considered collapsed if it contains at least one collapse event. The recovery rate is computed over collapsed trajectories as the fraction that recover from all collapse events by the final round.

Results are reported in Table 2. For frontier VLMs, RubSE reduces collapse for all three models (18.9% to 12.8% on average) and improves recovery for two (20.7% to 32.8% on average), suggesting that rubrics help strong code executors avoid over-broad repairs and recover from severe errors. For Qwen executors, the effect is mixed: average collapse increases from 14.8% to 18.1%, while average recovery improves from 23.5% to 30.2%, with only Qwen3-VL-32B showing a slight decline. Figure 6 further shows the distribution of rounds needed to recover from each collapse: for both open-source and frontier VLMs, RubSE concentrates in earlier rounds, requiring fewer recovery rounds on average. Overall, rubrics provide a stabilizing visual-repair context: they broadly improve recovery rate and speed, and substantially reduce collapse when paired with frontier executors.

5.3 Do Stronger Rubrics Transfer to Weaker Code Improvers?

To examine how rubric quality affects iterative self-evolution, we keep each Qwen model as the code improver but replace its self-generated rubrics with those generated and selected by GPT-5.4, the strongest VLM in our experiments. As shown in Figure 5, GPT-generated rubrics consistently outperform self-generated rubrics on UI2Code-Real across all three Qwen models, yielding stronger and more stable iterative gains. This suggests that

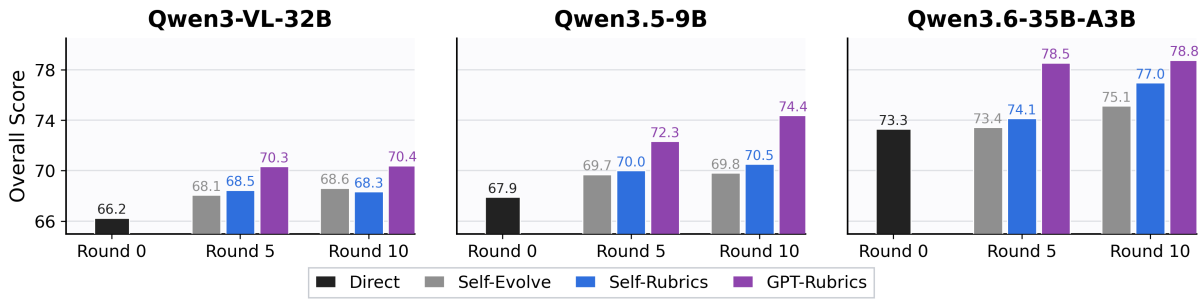


Figure 5: Results of applying GPT-generated rubrics for open-source VLM self-evolution. Across all three Qwen models, rubrics generated and selected by GPT-5.4 lead to stronger and more consistent gains than self-generated rubrics, particularly in early refinement rounds.

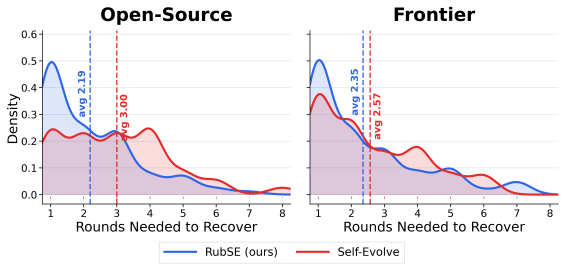


Figure 6: Density of rounds needed for recovery. Results are reported over all recovered trajectories. Dotted lines denote the average number of rounds required to recover from a collapse for each method.

Dataset	Rubrics	Tok.	Global	CSS	Numeric	Space	Style	Action
UI2C	Qwen	56.5	71%	50%	35%	20%	40%	50%
	GPT	57.3	89%	51%	7%	30%	41%	77%
D2C	Qwen	56.9	68%	70%	43%	34%	42%	65%
	GPT	54.8	91%	53%	7%	34%	51%	80%
D2C-H	Qwen	53.4	70%	65%	46%	26%	47%	58%
	GPT	54.6	90%	54%	7%	35%	49%	82%

Table 3: Content analysis of GPT- and Qwen-generated rubrics. "Tok." denotes the average number of words per rubric. Values are the percentage of rubrics containing the corresponding type of visual repair.

high-quality rubrics can serve as model-agnostic visual-repair context: even when the code improver itself is weaker, better external guidance can effectively steer refinement toward more localized and reliable visual corrections.

GPT rubrics operate at a larger and more actionable visual-repair scale. We analyze self-generated and GPT-generated rubrics across three Qwen models and three benchmarks, with numerical results reported in Table 3 and qualitative examples shown in Table 12. Self-rubrics produced by Qwen models are typically more low-level and narrowly scoped: they often prescribe concrete patches, naming CSS properties, pixel values, colors, or individual widgets, as reflected by their higher use of CSS/HTML cues on Design2Code (70% vs. 53%) and numeric or color constants (35–46% vs. 7%). In contrast, GPT-generated rubrics abstract away from single CSS fixes and instead focus on target perceptual relations, such as layout, scale, hierarchy, and spatial relationships between regions. Quantitatively, they contain more global-structure cues across all datasets (89–91% vs. 68–71%) and more action-oriented revision language (77–82% vs. 50–65%). This difference in *feedback granularity* helps explain Figure 5: GPT-generated rubrics transfer effectively to weaker Qwen code improvers, producing larger early gains by steering them beyond local CSS-level repair loops toward higher-level structural and actionable edits. Self-generated rubrics are still useful for Qwen models, but mainly through low-level edits that yield small yet consistent stepwise improvements.

6 Conclusion

In this paper, we introduced RubSE, a rubric-guided self-evolution framework for UI-to-code generation that instantiates visual-repair context as structured rubrics. Each rubric defines a focused repair target, identifies the corresponding visual mismatch, and specifies a possible correction direction. To construct visual-repair context at each iteration round, RubSE uses an EVOLVE–SELECT–HISTORY loop over rubrics: it explores candidate repair directions, selects one prioritized target, and carries previous rubrics forward to discourage repeated or over-broad revisions. This design preserves the flexibility of open-ended code revision while making iterative refinement more controlled and less prone to repeated repairs. Experiments across frontier and open-source vision-language models show that RubSE improves both final-round stability and best-round performance over naïve self-evolution. Further analyses show that rubric-guided repair helps recover from severe

visual regressions, and that high-quality rubrics can transfer from stronger rubric generators to weaker code improvers. Overall, RubSE provides a simple and practical mechanism to make test-time UI-to-code self-evolution more structured and reliable, paving the way for self-improving systems for complex generation tasks with coupled final artifacts.

Limitations

First, our experiments primarily evaluate three leading frontier models and three representative open-source variants on HTML/CSS webpage generation, leaving broader model families and UI languages underexplored. Second, RubSE does not explicitly guarantee code correctness during self-evolution. As a result, some refinement steps may introduce syntax or runtime errors, leading to rendering failures. Incorporating an explicit debugging loop into self-evolution would be a promising direction. Third, although VLM-based evaluation has improved, its judgments are not perfectly stable and may diverge from human evaluations in a minority of cases, particularly when assessing subtle visual differences in later refinement rounds. Our human study (Appendix B.1) supports the main comparison on 60 UI2Code-Real samples for two models. Developing more robust visual graders remains an important direction for future work.

References

Afra Feyza Akyürek, Advait Gosai, Chen Bo Calvin Zhang, Vipul Gupta, Jaehwan Jeong, Anisha Gunjal, Tahseen Rabbani, Maria Mazzone, David Randolph, Mohammad Mahmoudi Meymand, et al. 2025. Prbench: Large-scale expert rubrics for evaluating high-stakes professional reasoning. *arXiv preprint arXiv:2511.11562*.

Anthropic. 2025a. Claude code. https://code.claude.com/docs.

Anthropic. 2025b. Introducing claude sonnet 4.5. https://www.anthropic.com/news/claude-sonnet-4-5.

Rahul K Arora, Jason Wei, Rebecca Soskin Hicks, Preston Bowman, Joaquin Quiñonero-Candela, Foivos Tsimpourlas, Michael Sharman, Meghan Shah, Andrea Vallone, Alex Beutel, et al. 2025. Healthbench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775*.

Alisson Azzolini, Junjie Bai, Hannah Brandon, Jiaxin Cao, Prithvijit Chattopadhyay, Huayu Chen, Jinju Chu, Yin Cui, Jenna Diamond, Yifan Ding, et al. 2025. Cosmos-reason1: From physical common sense to embodied reasoning. *arXiv preprint arXiv:2503.15558*.

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. 2025. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*.

Tony Beltramelli. 2018. pix2code: Generating code from a graphical user interface screenshot. In *Proceedings of the ACM SIGCHI symposium on engineering interactive computing systems*, pages 1–6.

Tanvir Bhathal and Asanshay Gupta. 2025. Websight: A vision-first architecture for robust web agents. *arXiv preprint arXiv:2508.16987*.

Tommaso Calò and Luigi De Russis. 2026. Webui-95: A large-scale dataset of normalized web interfaces via ui-to-code generation. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*, pages 1–5.

Guo Chen, Zhiqi Li, Shihao Wang, Jindong Jiang, Yicheng Liu, Lidong Lu, De-An Huang, Wonmin Byeon, Matthieu Le, Max Ehrlich, et al. 2025a. Eagle 2.5: Boosting long-context post-training for frontier vision-language models. *Advances in Neural Information Processing Systems*, 38:91077–91100.

Ruibo Chen, Tianyi Xiong, Yihan Wu, Guodong Liu, Zhengmian Hu, Lichang Chen, Yanshuo Chen, Chenxi Liu, and Heng Huang. 2023. Gpt-4 vision on medical image classification–a case study on covid-19 dataset. *arXiv preprint arXiv:2310.18498*.

Yunnong Chen, Shixian Ding, YingYing Zhang, Wenkai Chen, Jinzhou Du, Lingyun Sun, and Liuqing Chen. 2025b. Designcoder: Hierarchy-aware and self-correcting ui code generation with large language models. *arXiv preprint arXiv:2506.13663*.

Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. 2024. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 24185–24198.

Jie Deng, Kaichun Yao, and Libo Zhang. 2026. Visrefiner: Learning from visual differences for screenshot-to-code generation. *arXiv preprint arXiv:2602.05998*.

Tong Ge, Yashu Liu, Jieping Ye, Tianyi Li, and Chao Wang. 2025. Advancing vision-language models in front-end development via data synthesis. *arXiv preprint arXiv:2503.01619*.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Nan Duan, Weizhu Chen, et al. 2024. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, volume 2024, pages 57734–57811.

Yi Gui, Zhen Li, Yao Wan, Yemin Shi, Hongyu Zhang, Bohua Chen, Yi Su, Dongping Chen, Siyuan Wu, Xing Zhou, et al. 2025a. Webcode2m: A real-world dataset for code generation from webpage designs. In *Proceedings of the ACM on Web Conference 2025*, pages 1834–1845.

Yi Gui, Zhen Li, Zhongyi Zhang, Guohao Wang, Tianpeng Lv, Gaoyang Jiang, Yi Liu, Dongping Chen, Yao Wan, Hongyu Zhang, et al. 2025b. Latcoder: Converting webpage design to code with layout-as-thought. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 721–732.

Yi Gui, Yao Wan, Zhen Li, Zhongyi Zhang, Dongping Chen, Hongyu Zhang, Yi Su, Bohua Chen, Xing Zhou, Wenbin Jiang, et al. 2025c. Uicopilot: Automating ui synthesis via hierarchical code generation from webpage designs. In *Proceedings of the ACM on Web Conference 2025*, pages 1846–1855.

Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. 2025. Rubrics as rewards: Reinforcement learning beyond verifiable domains. *arXiv preprint arXiv:2507.17746*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Jiaxin Huang, Shixiang Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. 2023. Large language models can self-improve. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 1051–1068.

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.

Yilei Jiang, Yaozhi Zheng, Yuxuan Wan, Jiaming Han, Qunzhong Wang, Michael R Lyu, and Xiangyu Yue. 2025. Screencoder: Advancing visual-to-code generation for front-end automation via modular multimodal agents. *arXiv preprint arXiv:2507.22827*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. 2023. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*.

Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, et al. 2024. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*.

Zhiyu Lin, Zhengda Zhou, Zhiyuan Zhao, Tianrui Wan, Yilun Ma, Junyu Gao, and Xuelong Li. 2025. Webuibench: a comprehensive benchmark for evaluating multimodal large language models in webui-to-code. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 15780–15797.

Chenxi Liu, Yanshuo Chen, Ruibo Chen, Tianyi Xiong, Tong Zheng, and Heng Huang. 2026. Prepare reasoning language models for multi-agent debate with self-debate reinforcement learning. *arXiv preprint arXiv:2601.22297*.

Chenxi Liu, Tianyi Xiong, Yanshuo Chen, Ruibo Chen, Yihan Wu, Junfeng Guo, Tianyi Zhou, and Heng Huang. 2025a. Modality-balancing preference optimization of large multimodal models by adversarial negative mining. *arXiv preprint arXiv:2506.08022*.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916.

Tianci Liu, Ran Xu, Tony Yu, Ilgee Hong, Carl Yang, Tuo Zhao, and Haoyu Wang. 2025b. Openrubrics: Towards scalable synthetic rubric generation for reward modeling and llm alignment. *arXiv preprint arXiv:2510.07743*.

Yuansheng Ni, Songcheng Cai, Xiangchao Chen, Jiarong Liang, Zhiheng Lyu, Jiaqi Deng, Kai Zou, Ping Nie, Fei Yuan, Xiang Yue, et al. 2025a. Viscoder2: Building multi-language visualization coding agents. *arXiv preprint arXiv:2510.23642*.

Yuansheng Ni, Ping Nie, Kai Zou, Xiang Yue, and Wenhu Chen. 2025b. Viscoder: Fine-tuning llms for executable python visualization code generation. *arXiv preprint arXiv:2506.03930*.

OpenAI. 2025a. Introducing codex. https://openai.com/index/introducing-codex/.

OpenAI. 2025b. Introducing gpt-5.2. https://openai.com/index/introducing-gpt-5-2/.

OpenAI. 2026. Introducing gpt-5.4. https://openai.com/index/introducing-gpt-5-4/.

MohammadHossein Rezaei, Robert Vacareanu, Zihao Wang, Clinton Wang, Bing Liu, Yunzhong He, and Afra Feyza Akyürek. 2025. Online rubrics elicitation from pairwise comparisons. *arXiv preprint arXiv:2510.07284*.

Rulin Shao, Akari Asai, Shannon Zejiang Shen, Hamish Ivison, Varsha Kishore, Jingming Zhuo, Xinran Zhao, Molly Park, Samuel G Finlayson, David Sontag, et al. 2025. Dr tulu: Reinforcement learning with evolving rubrics for deep research. *arXiv preprint arXiv:2511.19399*.

Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2025. Design2code:

Benchmarking multimodal code generation for automated front-end engineering. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3956–3974.

Davit Soselia, Khalid Saifullah, and Tianyi Zhou. 2023. Learning ui-to-code reverse generator using visual critic without rendering. *arXiv preprint arXiv:2305.14637*.

Pragya Srivastava, Harman Singh, Rahul Madhavan, Gandharv Patil, Sravanti Addepalli, Arun Suggala, Rengarajan Aravamudhan, Soumya Sharma, Anirban Laha, Aravindan Raghuveer, et al. 2025. Robust reward modeling via causal rubrics. *arXiv preprint arXiv:2506.16507*.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2025a. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919.

Xiyao Wang, Chunyuan Li, Jianwei Yang, Kai Zhang, Bo Liu, Tianyi Xiong, and Furong Huang. 2025b. Llava-critic-r1: Your critic model is secretly a strong policy model. *arXiv preprint arXiv:2509.00676*.

Yibin Wang, Yuhang Zang, Feng Han, Jiazi Bu, Yujie Zhou, Cheng Jin, and Jiaqi Wang. 2026. Unified personalized reward model for vision generation. *arXiv preprint arXiv:2602.02380*.

Zhilin Wang, Jaehun Jung, Ximing Lu, Shizhe Diao, Ellie Evans, Jiaqi Zeng, Pavlo Molchanov, Yejin Choi, Jan Kautz, and Yi Dong. 2025c. Profbench: Multi-domain rubrics requiring professional knowledge to answer and judge. *arXiv preprint arXiv:2510.18941*.

Jason Wu, Eldon Schoop, Alan Leung, Titus Barik, Jeffrey P Bigham, and Jeffrey Nichols. 2024. Uicoder: Finetuning large language models to generate user interface code through automated feedback. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7511–7525.

Tianyi Xiong, Yi Ge, Ming Li, Zuolong Zhang, Pranav Kulkarni, Kaishen Wang, Qi He, Zeying Zhu, Chenxi Liu, Ruibo Chen, et al. 2025a. Multi-crit: Benchmarking multimodal judges on pluralistic criteria-following. *arXiv preprint arXiv:2511.21662*.

Tianyi Xiong, Shihao Wang, Guilin Liu, Yi Dong, Ming Li, Heng Huang, Jan Kautz, and Zhiding Yu. 2026. Phycritic: Multimodal critic models for physical ai. *arXiv preprint arXiv:2602.11124*.

Tianyi Xiong, Xiyao Wang, Dong Guo, Qinghao Ye, Haoqi Fan, Quanquan Gu, Heng Huang, and Chunyuan Li. 2025b. Llava-critic: Learning to evaluate multimodal models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 13618–13628.

Mingde Xu, Zhen Yang, Wenyi Hong, Lihang Pan, Xinyue Fan, Yan Wang, Xiaotao Gu, Bin Xu, and Jie Tang. 2025. Webvia: A web-based vision-language agentic framework for interactive and verifiable ui-to-code generation. *arXiv preprint arXiv:2511.06251*.

Zhen Yang, Wenyi Hong, Mingde Xu, Xinyue Fan, Weihan Wang, Jiele Cheng, Xiaotao Gu, and Jie Tang. 2026. Ui2code^N: Ui-to-code generation as interactive visual optimization. In *Proceedings of the 43rd International Conference on Machine Learning*.

Li S Yifei, Allen Chang, Chaitanya Malaviya, and Mark Yatskar. 2025. Researchqa: Evaluating scholarly question answering at scale across 75 fields with survey-mined questions and rubrics. *arXiv preprint arXiv:2509.00496*.

Tianyu Yu, Zefan Wang, Chongyi Wang, Fuwei Huang, Wenshuo Ma, Zhihui He, Tianchi Cai, Weize Chen, Yuxiang Huang, Yuanqian Zhao, et al. 2025. Minicpm-v 4.5: Cooking efficient mllms via architecture, data, and training recipe. *arXiv preprint arXiv:2509.18154*.

Youliang Yuan, Qiuyang Mang, Jingbang Chen, Hong Wan, Xiaoyuan Liu, Junjielong Xu, Jen-tse Huang, Wenxuan Wang, Wenxiang Jiao, and Pinjia He. 2025. Curing miracle steps in llm mathematical reasoning with rubric rewards. *arXiv preprint arXiv:2510.07774*.

Sukmin Yun, Haokun Lin, Rusiru Thushara, Mohammad Q Bhat, Yongxin Wang, Zutao Jiang, Mingkai Deng, Jinhong Wang, Tianhua Tao, Junbo Li, et al. 2024. Web2code: A large-scale webpage-to-code dataset and evaluation framework for multimodal llms. *Advances in neural information processing systems*, 37:112134–112157.

Chenchen Zhang, Yuhang Li, Can Xu, Jiaheng Liu, Ao Liu, Changzhi Zhou, Ken Deng, Dengpeng Wu, Guanhua Huang, Kejiao Li, et al. 2025a. Artifactsbench: Bridging the visual-interactive gap in llm code generation evaluation. *arXiv preprint arXiv:2507.04952*.

Di Zhang, Jingdi Lei, Junxian Li, Xunzhi Wang, Yujie Liu, Zonglin Yang, Jiatong Li, Weida Wang, Suorong Yang, Jianbo Wu, et al. 2025b. Critic-v: Vlm critics help catch vlm errors in multimodal reasoning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9050–9061.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. 2025c. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*.

Xinlu Zhang, Yujie Lu, Weizhi Wang, An Yan, Jun Yan, Lianke Qin, Heng Wang, Xifeng Yan, William Yang Wang, and Linda Ruth Petzold. 2023. Gpt-4v (ision) as a generalist evaluator for vision-language tasks. *arXiv preprint arXiv:2311.01361*.

Shitian Zhao, Haoquan Zhang, Shaoheng Lin, Ming Li, Qilong Wu, Kaipeng Zhang, and Chen Wei. 2025a. Pyvision: Agentic vision with dynamic tooling. *arXiv preprint arXiv:2507.07998*.

Xuanle Zhao, Deyang Jiang, Zhixiong Zeng, Lei Chen, Haibo Qiu, Jing Huang, Yufeng Zhong, Liming Zheng, Yilin Cao, and Lin Ma. 2025b. Vincicoder: Unifying multimodal code generation via coarse-to-fine visual reinforcement learning. *arXiv preprint arXiv:2511.00391*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623.

Ting Zhou, Yanjie Zhao, Xinyi Hou, Xiaoyu Sun, Kai Chen, and Haoyu Wang. 2025. Declarui: Bridging design and development with automated declarative ui code generation. *Proceedings of the ACM on Software Engineering*, 2(FSE):219–241.

A Evaluation Protocol

In experiments, we evaluate generated code quality by comparing the rendered image against the target image under the VLM-as-a-Judge paradigm. We use VLM judges in two complementary ways:

- **Overall Rating**: We adapt the overall-rating judge from Yang et al. (2026), in which a VLM evaluator assigns a single overall score on a 100-point scale in one pass. Their study validates this judge by demonstrating strong agreement with human judgments. The evaluation prompt is provided in Table 4.
- **Aspect-level Rating**: As shown in Table 5, we define five aspects of visual fidelity. For each aspect, the VLM judge independently assigns a Likert-scale score from 1 to 7, and we report the mean score across aspects as the final aspect-level metric. This aspect-level evaluation provides a more fine-grained measure of localized visual changes during iterative refinement. As a sanity check, we manually inspected 60 rendered-image pairs from consecutive iterative rounds. The direction of model-based aspect-level score changes matched manual judgments in 47 cases, suggesting a useful approximate signal for tracking localized refinement trends.

Note that intermediate Likert-scale anchors are used only for the aspect-level ratings, not for the overall rating. The overall rating provides a holistic assessment across layout, spacing, typography, styling, and completeness. Because overall quality may reflect different combinations of errors, intermediate anchors would require prescribing potentially arbitrary trade-offs across these heterogeneous aspects. In contrast, each aspect-level rating isolates a single visual dimension, allowing intermediate anchor descriptions to be more clearly defined and helping to improve rating consistency. As a sanity check, the overall score correlates well with the anchored aspect mean (Pearson's $r = 0.67$; Spearman's $\rho = 0.74$).

Why not CLIP evaluation. We do not include the CLIP-based evaluation used in Si et al. (2025). CLIP similarities are useful for capturing coarse overall quality differences across generation models, but are often insensitive to the smaller, localized changes produced by iterative self-evolution. To verify this, we apply `google/siglip2-large-patch16-512` to compute image similarity and manually inspect 40 pairs of consecutive iterative rounds with clear aspect-level performance changes. In 17 cases, the CLIP score either remained nearly unchanged or showed a trend opposite to the human-observed visual improvement or degradation, suggesting that CLIP-based metrics are not sufficiently informative for analyzing fine-grained refinement behavior.

Evaluation Prompt (Overall)

I will provide you with two UI images. The first is the reference UI screenshot; the second is a generated image rendered from VLM-produced code. Evaluate the visual fidelity of the generated image with respect to the reference, scoring from 0 to 100 (0 = not faithful at all, 100 = perfectly faithful).

Consider the following aspects:

- Layout & geometry: structure, alignment, relative positions, and element sizes.
- Spacing: padding, margins, gaps, line-heights, and overall visual density.
- Typography: font sizes/weights, hierarchy, text placement, and line breaks.
- Styling: colors, backgrounds, borders, corner radius, shadows, and button styles.
- Completeness: all visible UI components are present with no omissions or hallucinations.

First provide your reasoning, then output the final score inside LaTeX `\boxed{}`.

Table 4: Evaluation prompt for overall score.

B Additional Experiments and Analyses

B.1 Human Evaluation

We conduct an additional human evaluation on 60 samples from UI2Code-Real, comparing the round-10 outputs of RubSE and naïve self-evolution generated by GPT-5.2 and GPT-5.4. Following the same overall-quality evaluation criteria as those specified in Table 4, two authors first evaluate each pair independently. They then resolve disagreements through discussion to assign each pair one of three consensus labels: a RubSE win, a tie, or a naïve self-evolution win.

As shown in Table 6, RubSE is clearly preferred over naïve self-evolution for both GPT-5.2 and GPT-5.4. Two-sided exact sign tests on the non-tied comparisons confirm that this preference is statistically significant for both generation models. This provides complementary evidence that the improvements measured by the automated evaluators correspond to perceptible gains in output quality.

Evaluate Prompt (Aspect)

You will be shown two UI images. The first is the reference UI screenshot; the second is a generated image rendered from VLM-produced code. Evaluate the visual fidelity of the generated image with respect to the reference, for the following aspect ONLY.

Aspect: `{aspect_name}`
Definition: `{aspect_description}`

Scoring: Use a 1–7 Likert scale (1 = not faithful at all, 7 = perfectly faithful).

Score standards (Likert 1–7):

- 7: Perfect match for this aspect; no noticeable differences.
- 6: Very close match; only tiny, hard-to-notice differences.
- 5: Mostly matches; small differences are noticeable but do not change the overall impression.
- 4: Mixed quality; clear differences that moderately affect the aspect.
- 3: Weak match; major differences are obvious and significantly affect the aspect.
- 2: Very poor match; severe differences dominate this aspect.
- 1: Completely unfaithful for this aspect.

Important notes:

- Evaluate only the specified aspect; do NOT consider other aspects.
- Ignore differences in external photographic content that cannot be faithfully reproduced using HTML/CSS.

Output format (strict): Return a single JSON object ONLY, with keys:

- `aspect`: string, must equal the Aspect name above.
- `rationale`: justification focusing only on the specified aspect.
- `score`: integer in [1, 7].

Aspect List:

- `layout_geometry`: Layout & geometry: correctness of the overall page structure, including major regions (e.g., header, sidebar, main content), element alignment, relative positioning, and approximate element sizes.
- `spacing_density`: Spacing & density: consistency of padding, margins, gaps, and line-heights, as well as whether the overall visual density matches the reference UI.
- `typography_text`: Typography & text hierarchy: faithfulness of font sizes, font weights, hierarchical relationships between titles and body text, text placement, and line breaks.
- `styling_visual`: Visual styling: accuracy of colors, backgrounds, borders, corner radius, shadows, dividers, and button or card styling.
- `completeness`: Content completeness: whether all visible UI elements in the reference are present in the generated UI, with no missing components and no hallucinated or extraneous elements.

Table 5: Evaluation prompt for aspect-level scoring.

Model	#RubSE	#Tie	#Naïve	Win	Win+Tie	p-value
GPT-5.2	39	4	17	65.0%	71.7%	0.0046
GPT-5.4	35	10	15	58.3%	75.0%	0.0066

Table 6: Human pairwise evaluation of the round-10 outputs produced by RubSE and naïve self-evolution on UI2Code-Real. Win rates and win-plus-tie rates are computed over all 60 comparisons for each model. The reported p-values are obtained using two-sided exact sign tests that exclude ties.

B.2 Judge-Score Variability

To assess evaluation variability, we re-evaluate the initial and round-10 outputs of all models using three additional independent judge runs, separate from those used for the main results. Table 7 reports the mean $\pm$ standard deviation of the round-10 improvement over the corresponding initial generation across these three runs.

The relative ordering of RubSE and naïve self-evolution in these additional runs matches the main results (Table 1) in every setting: RubSE achieves a larger mean improvement in all nine frontier-model settings and in seven of the nine open-source-model settings, with both exceptions occurring on Qwen3-VL-32B-Instruct, whose behavior is analyzed in Appendix B.4. Standard deviations range from 0.09 to 0.82 points and are generally smaller for frontier models. Overall, these additional runs reproduce the comparative pattern in the main results.

B.3 Computational Cost

We analyze the computational cost of RubSE for both API-based and open-source models using 50 randomly selected UI2Code-Real samples over 10 refinement rounds (500 rounds in total). All reported values are averaged per refinement round.

API-based cost. We report the average per-round input and output token counts and API cost of GPT-5.4 based on OpenAI's standard rates of $2.50 per million input tokens and $15 per million output tokens. As shown in Table 8, although RubSE makes three model calls per refinement round, EVOLVE and SELECT generate relatively short outputs. Compared with naïve self-evolution, RubSE uses $3.16\times$ as many input tokens but only $1.21\times$ as many output tokens, resulting in $2.38\times$ as many total tokens and $1.60\times$ the API cost.

Open-source inference latency. For the open-source setting, we measure average per-round inference time using Qwen3.5-9B served with `vLLM` on a single H200 GPU with a batch size of 1. The first

Model	Method	UI2Code-Real	Design2Code	Design2Code-HARD
GPT-5.4	Self-Evolve	+0.39±0.13	−1.28±0.09	−1.98±0.59
	RubSE	+2.87±0.29	+0.98±0.26	+0.70±0.35
GPT-5.2	Self-Evolve	+0.65±0.22	−2.54±0.22	−0.29±0.21
	RubSE	+3.32±0.28	+0.72±0.10	+1.47±0.51
Claude-Sonnet-4.5	Self-Evolve	+2.25±0.18	+0.44±0.58	+0.37±0.34
	RubSE	+4.66±0.31	+0.57±0.39	+0.76±0.30
Qwen3-VL-32B	Self-Evolve	+2.15±0.60	+1.59±0.14	+5.68±0.63
	RubSE	+2.01±0.61	+1.76±0.12	+3.67±0.74
Qwen3.5-9B	Self-Evolve	+1.74±0.12	+3.28±0.23	+4.47±0.70
	RubSE	+2.53±0.12	+4.82±0.09	+5.29±0.64
Qwen3.6-35B-A3B	Self-Evolve	+1.98±0.23	+1.08±0.26	+0.66±0.67
	RubSE	+4.27±0.82	+1.67±0.31	+1.91±0.59

Table 7: Round-10 improvement in overall score over the initial generation, reported as mean ± std over three additional independent GPT-5.2 judge runs. Across all models and benchmarks, the relative ordering of RubSE and naïve self-evolution in these additional runs is consistent with that reported in the main results (Table 1).

Method	# Input	# Output	# Total	Cost (¢)
Naïve Self-Evolve	5,912.0	3,971.0	9,883.0	7.4
RubSE EVOLVE	6,103.8	605.3	6,709.1	2.4
RubSE SELECT	6,517.1	85.4	6,602.5	1.8
RubSE IMPROVE	6,059.2	4,131.5	10,190.7	7.7
RubSE total	18,680.1	4,822.2	23,502.3	11.9

Table 8: Average per-round token usage and API cost on 50 UI2Code-Real samples over 10 refinement rounds using GPT-5.4. "#" denotes the number of tokens, and cost is reported in cents (¢).

Method	# Input	# Output	# Total	Time (s)
Naïve Self-Evolve	9,025.4	3,827.7	12,853.1	102.7
RubSE EVOLVE	9,215.4	487.5	9,702.9	13.2
RubSE SELECT	9,510.5	81.3	9,591.8	2.4
RubSE IMPROVE	9,113.8	3,423.9	12,537.7	89.7
RubSE total	27,839.7	3,992.7	31,832.4	105.3

Table 9: Average per-round token usage and inference time for Qwen3.5-9B. Results exclude the first two samples, which are used to warm up the vLLM inference engine. "#" denotes the number of tokens.

two samples are excluded to allow the inference engine to warm up. As shown in Table 9, RubSE takes 105.3 seconds per refinement round, compared with 102.7 seconds for naïve self-evolution, corresponding to only 2.5% additional latency. In this setting, inference time is dominated by autoregressive decoding of the long HTML output rather than prompt prefilling; consequently, the short-output EVOLVE and SELECT calls introduce relatively little additional latency.

Considering both effectiveness and efficiency, RubSE increases monetary cost to 1.60× with GPT-5.4 while mitigating the late-round degradation associated with visual repair coupling. With Qwen3.5-9B, RubSE incurs only 2.5% additional latency while achieving 1.4× the average round-10 performance gain on the selected 50-sample subset. This moderate additional cost is spent on structuring the repair context rather than on generating more code, and is accompanied by more stable and effective self-evolution.

Separation of EVOLVE and SELECT. As discussed in Section 3.3, issuing EVOLVE and SELECT as separate calls keeps candidate generation and prioritization as distinct steps. We additionally evaluate a variant that merges EVOLVE and SELECT into a single request on the same 50-sample subset. The merged call uses an average of 6,168.8 input tokens and 641.7 output tokens per round, costing 2.5 cents. By comparison, the separate EVOLVE and SELECT calls cost 2.4 and 1.8 cents, respectively, as reported in Table 8. Merging the two modules reduces the total per-round cost by 14.2%, but decreases the average round-10 improvement in overall score by 0.5 points.

B.4 Understanding the Exception of Qwen3-VL-32B-Instruct

To better understand why Qwen3-VL-32B-Instruct does not consistently benefit from RubSE compared with naïve self-evolution, we analyze the distribution of rubric types across Qwen models during the EVOLVE → SELECT procedure. As reported in Table 10, Qwen3-VL-32B-Instruct gener-

Model	Rubric-gen.	Spacing density	Completeness
Qwen3-VL	Self	27.1% → 13.6%	2.3% → 4.5%
Qwen3.5	Self	12.8% → 4.8%	8.1% → 10.0%
Qwen3.6	Self	8.8% → 3.6%	9.2% → 8.9%
Qwen3-VL	GPT-5.4	16.6% → 8.7%	7.7% → 8.0%

Table 10: Rubric-type distributions produced by EVOLVE and retained by SELECT. "Model" denotes the model used for visual-code refinement, while "Rubric-gen." denotes the model used to generate and select rubrics. Each cell reports EVOLVE → SELECT.

ates substantially more spacing-density rubrics and fewer completeness rubrics than Qwen3.5-9B and Qwen3.6-35B-A3B, and SELECT only partially corrects this imbalance. Consequently, its selected targets remain overly focused on local adjustments, such as margins and line height, while underemphasizing missing or extraneous components, which can trap refinement in repeated local edits. Using GPT-5.4-generated rubrics substantially reduces this imbalance and clearly outperforms naïve self-evolution (Figure 5). These results suggest that the performance gap primarily originates from biased rubric generation that SELECT does not fully correct, rather than from the model's ability to perform rubric-guided improvement.

C Case Study

- In Table 11, we provide a trajectory-level comparison between RubSE and naïve self-evolution on the same target screenshot. RubSE yields more controlled and consistent improvements, whereas naïve self-evolution oscillates across rounds, repeatedly altering already-correct regions and exhibiting stronger visual repair coupling.

- In Table 12, we present three selected examples contrasting GPT-generated rubrics with self-generated rubrics for the same previous-round Qwen outputs. These examples illustrate the granularity difference behind the quantitative trends in Table 3: stronger external rubrics provide broader and more structural guidance.

D Prompts for Self-Evolution

We include the detailed prompt for naïve self-evolution in Table 13. For RubSE, the prompts for rubric generation, rubric selection, and rubric-guided single-step self-evolution are provided in Tables 14, 15, and 16, respectively.

E Additional Discussion

Relationship to General-Purpose Coding Agents. Modern general-purpose coding agents, such as Claude Code (Anthropic, 2025a), Codex (OpenAI, 2025a), and OpenHands (Wang et al., 2025a), provide execution environments that define actions available to the model, including file access, command execution, screenshot inspection, and iterative code editing. RubSE operates at a different and complementary level: it defines an explicit, model-agnostic policy that specifies how rendered feedback is structured, prioritized, and retained to guide visual repair through its typed rubric taxonomy and EVOLVE–SELECT–HISTORY procedure. In our experiments, we implement it through constrained, role-separated calls to both open-source and frontier models, without requiring a general-purpose agent harness. This design isolates the policy-level contribution: rubric-guided repair is compared with unstructured self-evolution under the same model and execution setting.

Artifact Use and Licensing. We use UI2Code under its MIT license and Design2Code under its ODC-By license. These benchmarks consist of publicly available UI screenshots or rendered webpages; we do not collect new user data. For models, we use the Qwen3-VL series under its Apache-2.0 license, while proprietary frontier models, including GPT and Claude, are accessed through their respective API/service terms. All artifacts are used only for academic evaluation, and we do not redistribute third-party datasets, model weights, or proprietary model outputs.

Documentation of Artifacts. We report the benchmark and model details in Section 4.1. These benchmarks provide screenshots as visual inputs for VLMs to generate HTML/CSS for webpage reconstruction. Since our evaluation focuses on UI screenshots and webpage-to-code generation, demographic attributes and linguistic phenomena are not applicable.

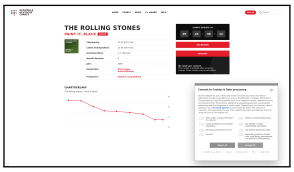
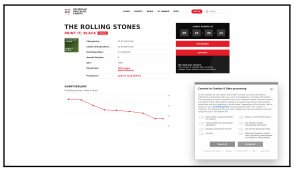
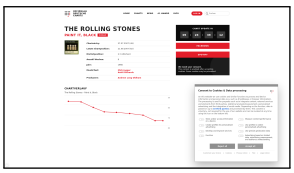
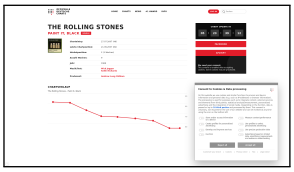
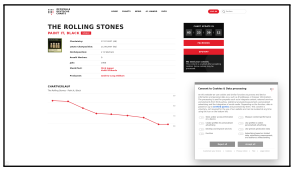
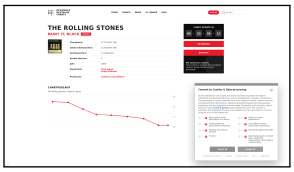
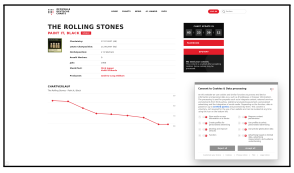
Round	Self-Evolve	Selected Rubric	RubSE
GT		–	
0		–	
1		**Cookie modal size and placement** (`layout_geometry`). The reference cookie dialog is larger and anchored lower/right, covering a big portion of the sidebar and chart area; the current modal is smaller and placed too high (top:260px) and not as far right. Increase modal width/height and reposition it to bottom-right (e.g., position:fixed; right:70px; bottom:40px) to match the overlay behavior and coverage seen in the target.	
2		**Top navigation horizontal alignment** (`layout_geometry`). In the reference, the main nav (HOME, CHARTS, NEWS, #1 AWARD, INFO) sits right-of-center near the top, while the LOG-IN + search are at the far top-right; in the current render, the nav is centered and the right controls are pushed down the line. Adjust the topbar layout, e.g., three-column grid with fixed left brand, flexible middle nav aligned right, and right controls pinned to the edge, so the nav is not centered and matches the reference positioning.	
3		**Album cover artwork: use an image instead of gradient placeholder** (`completeness`). The reference shows the actual "Paint It, Black" cover image with detailed artwork. The current render uses a green striped placeholder; replace the cover background with an image or a background-image and size it to match the square cover presentation.	
4		**Page width and left content scaling** (`layout_geometry`). The reference has a wider overall canvas with more whitespace and a slightly more left-shifted main content block; the current layout feels more compressed due to a fixed 1200px wrap and heavy inner padding. Increase the max width or switch to a larger responsive max-width and adjust left/right padding so the main column proportions match the reference.	
5		**Right sidebar card widths and stacking** (`layout_geometry`). In the reference, the black countdown card, red FACEBOOK bar, and red SPOTIFY bar are narrower and aligned as a single column with more whitespace to the right; in the current render they span the full 340px sidebar and look too wide. Reduce the sidebar column width and/or set a fixed card width with left alignment so the cards match the narrower block seen in the target.	
6		**Cookie toggle controls should be radio/slider style with red accent on right** (`styling_visual`). The reference shows compact toggle/radio-like controls with a red accent/knob on the right side of each control, while the current switches are gray with a white knob on the left and an '×' overlay. Redesign the toggle to place the knob/indicator on the right and use the red accent to match the reference control styling.	
7		**FACEBOOK bar text alignment** (`typography_text`). In the reference, the FACEBOOK label is left-aligned within the red bar with noticeable left padding; the current implementation centers the label. Change the FACEBOOK bar to left-align text and add left padding similar to the reference, roughly 18–24px.	

Table 11: Qualitative comparison on iterative rounds generated by GPT-5.2. Naïve self-evolution exhibits unstable refinement with oscillatory edits across rounds. In round 5, although it fixes the toggle color and top-bar length, it distorts the cookie modal's scale. This error is repeatedly modified in later rounds but remains unresolved. In contrast, RubSE follows a more structured repair trajectory guided by selected rubrics, progressively addressing the cookie modal, top bar, line-chart scale, and toggle color. Although RubSE still exhibits visual repair coupling when the chart-scale adjustment temporarily misplaces the cookie modal in round 4, it corrects the issue in the next round, leading to more controlled and recoverable gains.

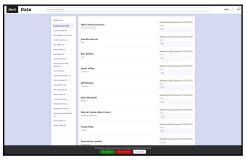
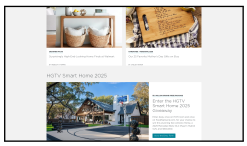
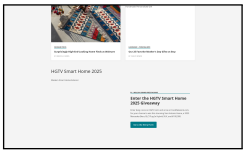
Model	GT	Previous	GPT-5.4 Rubric	Self Rubric
Qwen3-VL-32B-Instruct			**Main content container framing** (`styling_visual`). The central results area in the reference is a large flat light-gray panel with thin borders and no card shadow, while the current attempt uses a smaller white card with rounded corners and shadow. Expand the main panel vertically and horizontally, remove the shadow/radius, and use a subtle border so the list feels like a full-width table region.	**Correct count selector styling and layout** (`styling_visual`). The count selectors (e.g., '4 v') in the main content are currently rendered as rounded rectangular buttons with borders and hover effects, which do not match the target UI. In the target, they appear as small, compact inline elements with a blue number and a downward arrow inside a minimal white circle with a thin border. Update the '.count-selector' CSS to remove the background, border, and padding, and instead use a small circular container with a thin border and blue text.
Qwen3.5-9B			**Missing hero image in the lower-left feature area** (`completeness`). The large Smart Home photo is absent in the rendered attempt, leaving a blank area with only the image alt text visible. Ensure the lower section uses a reliably loading image and places it as a large left column visual, matching the reference where the house photo fills most of the left half of the feature row.	**Incorrect Category Label Styling** (`styling_visual`). In the target screenshot, the category labels (e.g., 'DESIGNER PICKS') are bold, uppercase sans-serif text with a thin, light blue underline. The current attempt applies a thick, dark teal border-bottom to these labels, which creates a heavy, incorrect visual style compared to the subtle underline in the reference.
Qwen3.6-35B-A3B			**Full-width centered page shell** (`layout_geometry`). The current page is constrained to a narrow 960px column and leaves a large empty gray area on the right, while the reference uses a much wider centered white canvas that spans most of the viewport. Increase the main page wrapper width to roughly 1100–1140px, center it within the light gray background, and keep the content and right appearance panel inside that single wide white area.	**Tab Active State Styling** (`styling_visual`). The 'Template' tab is currently styled as the active tab with a bold font and bottom border. In the target screenshot, the 'Discussion' tab is the active one. The active class should be moved to 'Discussion' and 'Template' should be inactive.

Table 12: Selected qualitative examples comparing GPT-generated and self-generated rubrics for Qwen self-evolution. For each Qwen code improver, we show the target screenshot, the initial rendered output, and the selected rubric used for the next refinement step. Rubrics from GPT-5.4 tend to identify higher-level visual-repair targets, such as page framing, missing major visual regions, or global layout structure. In contrast, self-generated rubrics often focus on more local implementation details, such as selector styling, underline thickness, or active-tab states.

Naïve Self-Evolution

You are an expert web developer specializing in HTML and CSS.

The user will provide:

- A target webpage screenshot (ground-truth UI).
- A screenshot/render of a previous HTML+CSS attempt.
- The previous attempt's HTML+CSS source.

Your task is to improve the previous HTML+CSS so that the rendered page matches the target screenshot more closely.

Output format requirements:

1. First, output a `<think>...</think>` block.
   - Compare the previous attempt against the target screenshot.
   - Identify mismatches and what to change: layout, spacing, typography, colors, alignment, and missing elements.
   - This is internal analysis; do not include any code here.
2. Then, output exactly one ```` ```html``` ```` code block.
   - The code block must contain a single complete HTML document.
   - Include all CSS inside `<style>` tags.
   - Do not use JavaScript.
   - Do not reference any external files, libraries, images, or fonts.
   - Use a fixed 1920×1080 viewport/aspect ratio. Set the root layout to exactly 1920px wide and 1080px tall, and avoid responsive rules that change this aspect ratio.

Critical: Do not output anything other than the `<think>` block followed immediately by the ```` ```html``` ```` block. No extra text before, between, or after them.

The first image is the target screenshot, the second image is the previous rendered UI. The HTML text is the previous attempt to improve.

Table 13: Prompt for naïve self-evolution.

RubSE: Rubric Generation

You are an expert UI evaluator and web developer.

The user will provide:

- A target webpage screenshot (ground-truth UI).
- A screenshot/render of a previous HTML+CSS attempt.
- The previous attempt's HTML+CSS source.
- Optionally, a JSON array of previously-addressed rubrics from earlier rounds; these are history items to avoid.

Your task is to produce a list of actionable, example-specific rubrics that identify the most important ways the HTML/CSS should be improved so its rendered page matches the target screenshot more closely. If rubric history is provided, generate only new rubrics for remaining mismatches and do not restate, rephrase, or slightly vary historical rubrics.

Each rubric must be assigned one type from this fixed set:

- `layout_geometry`: page structure, major regions, alignment, relative positioning, and element sizes.
- `spacing_density`: padding, margins, gaps, line-heights, and overall visual density.
- `typography_text`: font sizes, font weights, text hierarchy, text placement, and line breaks.
- `styling_visual`: colors, backgrounds, borders, corner radius, shadows, dividers, and button or card styling.
- `completeness`: missing visible UI elements, hallucinated components, and extraneous elements.

Rubric design requirements:

- Each rubric must be concrete, checkable, and specific to this example.
- Clearly describe what is currently mismatched and how it should be corrected.
- Ground every rubric in observable evidence from the provided images and HTML.
- Avoid generic or reusable UI advice.
- Produce 3–8 rubrics, ordered roughly by importance.

Output only a single JSON array. Each element must be a JSON object with exactly three keys: `title`, `description`, and `type`. Do not include commentary, markdown, code blocks, or text outside the JSON array.

Table 14: Prompt for rubric generation in RubSE.

RubSE Rubric Selection

You are an expert UI evaluator and web developer.
The user will provide:

- A target webpage screenshot (ground-truth UI).
- A screenshot/render of a previous HTML+CSS attempt.
- The previous attempt's HTML+CSS source.
- A JSON array of candidate rubrics, each with `title`, `description`, and `type`.

Your task is to select the single most critical rubric from the candidate list: the one that, if addressed, would produce the greatest improvement in visual similarity between the rendered page and the target screenshot.
Selection criteria:

- Prioritize issues that are visually prominent and immediately apparent.
- Prefer rubrics where fixing the issue requires a targeted, well-scoped HTML/CSS change.
- Consider visual impact relative to implementation effort.
- Do not select a rubric that is already well-resolved in the previous attempt.

Output only a single JSON object with exactly three keys: `title`, `description`, and `type`. The object must be copied exactly from the candidate list. Do not include commentary, markdown, code blocks, or text outside the JSON object.

Table 15: Prompt for rubric selection in RubSE.

RubSE: Rubric-Guided Improvement

You are an expert web developer specializing in HTML and CSS.
The user will provide:

- A target webpage screenshot (ground-truth UI).
- A screenshot/render of a previous HTML+CSS attempt.
- The previous attempt's HTML+CSS source.
- A JSON object containing the selected improvement rubric.

Your task is to improve the previous HTML+CSS so that the rendered page matches the target screenshot more closely.
Output format requirements:

1. First, output a `<think>...</think>` block.
   - Use the provided rubric as a checklist.
   - Compare the previous attempt against the target screenshot.
   - Identify the top mismatches and propose concrete HTML/CSS changes.
   - This is internal analysis; do not include any code here.
2. Then, output exactly one ```` ```html``` ```` code block.
   - The code block must contain a single complete HTML document.
   - Include all CSS inside `<style>` tags.
   - Do not use JavaScript.
   - Do not reference any external files, libraries, images, or fonts.
   - Use a fixed 1920×1080 viewport/aspect ratio. Set the root layout to exactly 1920px wide and 1080px tall, and avoid responsive rules that change this aspect ratio.

Critical: Do not output anything other than the `<think>` block followed immediately by the ```` ```html``` ```` block. No extra text before, between, or after them.

Table 16: Prompt for single-step rubric-guided code improvement in RubSE.